ADAMA SCIENCE & TECHNOLOGY UNIVERSITY

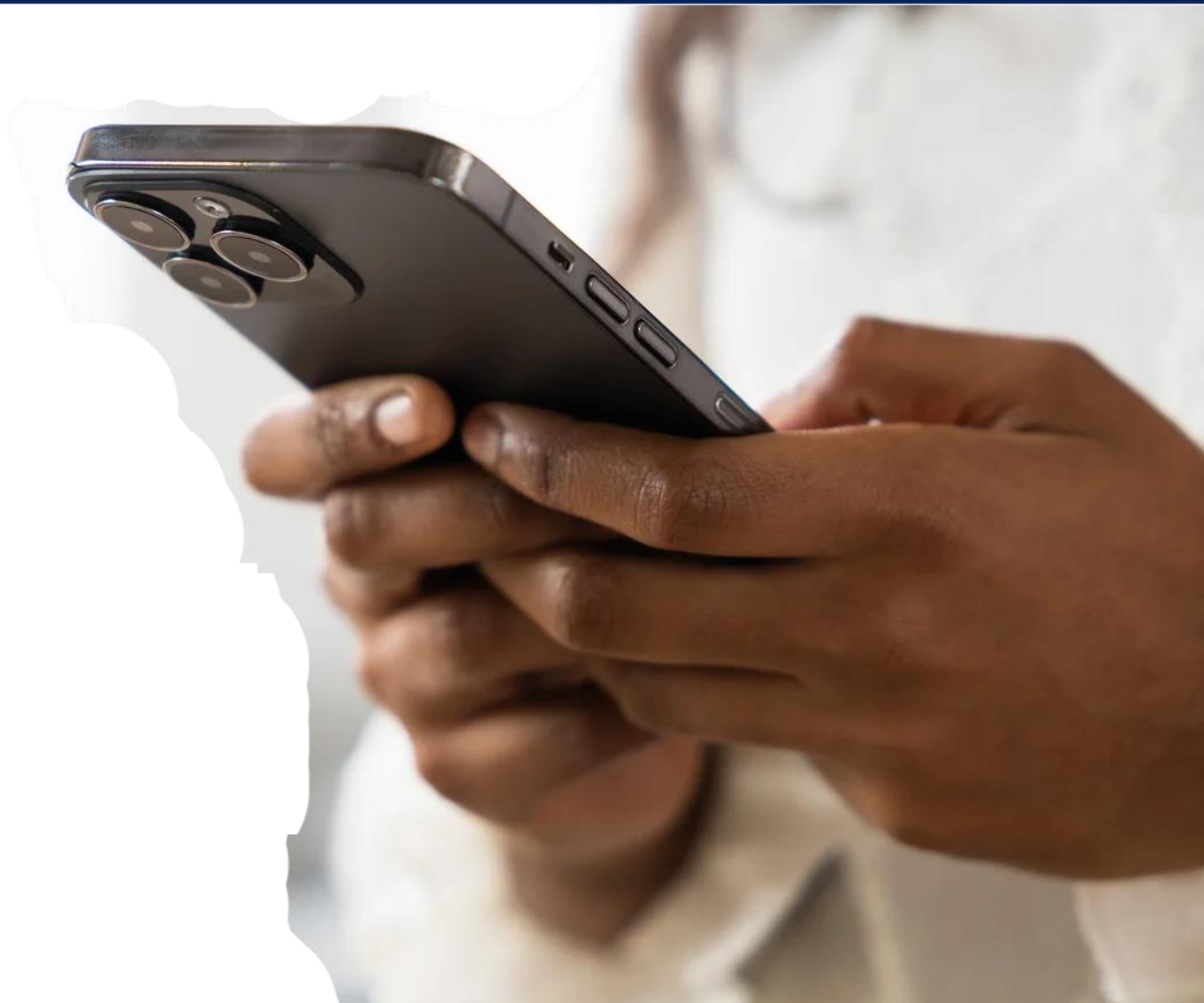
CHAPTER 2

Mobile Operating System

What is a Mobile Operating System?

Core Definition:

Software that manages hardware resources and provides common services for applications , the **intermediary** between users, apps, and hardware.



What is a Mobile Operating System?

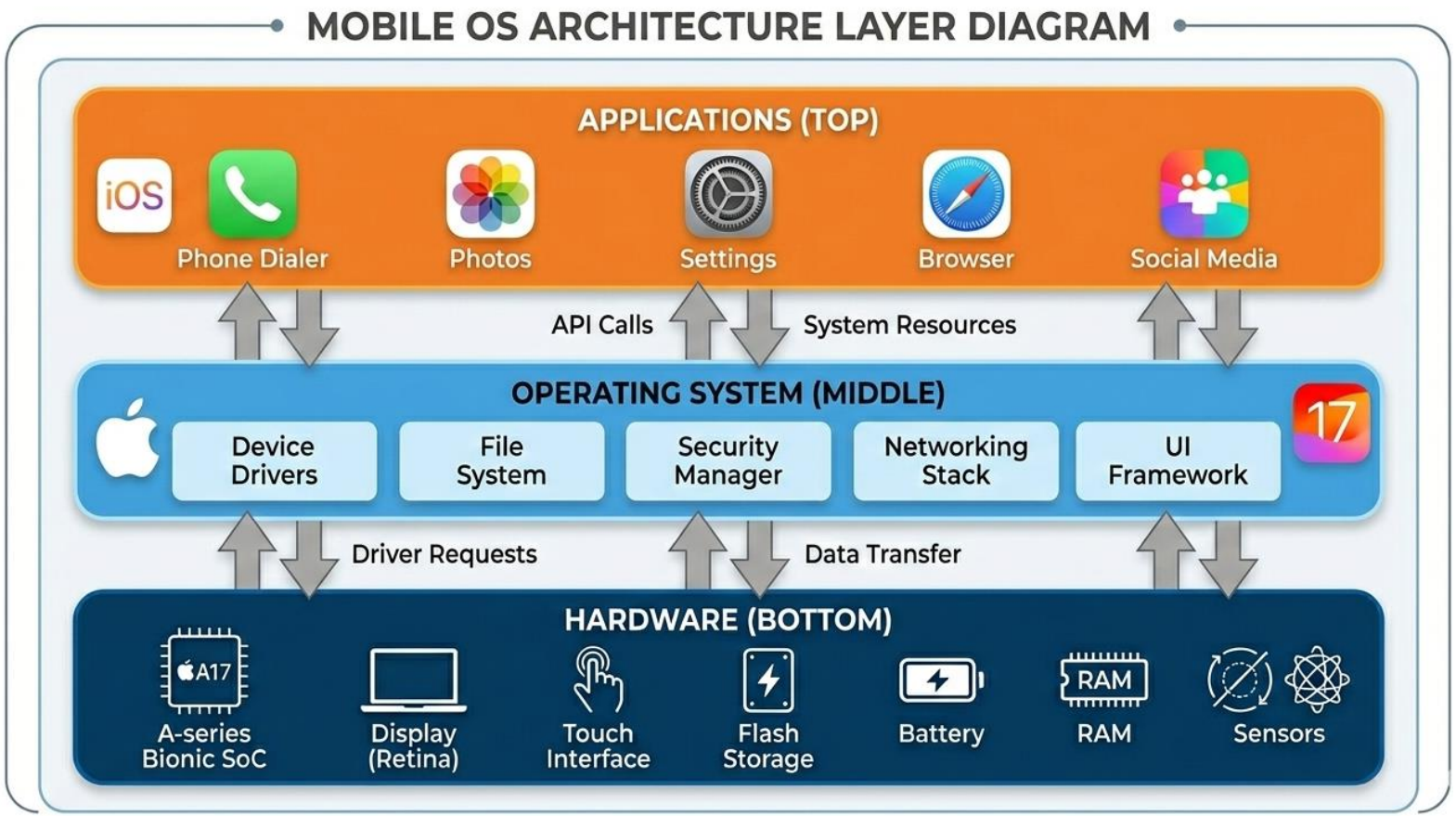
Key Responsibilities:

Function	Description
Hardware Management	CPU scheduling, memory allocation, power optimization
Security & Privacy	Sandboxing, permissions, encryption
User Interface	Touch handling, animations, accessibility
App Lifecycle	Launching, pausing, resuming, terminating apps
Connectivity	Managing cellular, Wi-Fi, Bluetooth, NFC
Development Framework	APIs and tools for building apps

What is a Mobile Operating System?

Analogy:

- **Hardware** = Car engine and chassis
- **Operating System** = Driver + dashboard + controls
- **Applications** = Passengers giving directions



The Three Major Players — Overview

The Three Major Players — Overview

Attribute	**Android**	**iOS**	**HarmonyOS**
Developer	Google	Apple	Huawei
Initial Release	September 2008	June 2007	August 2019
Base Architecture	Linux kernel (macro-kernel)	Darwin/XNU (hybrid kernel)	Distributed micro-kernel
Open Source	Yes (AOSP)	No	OpenHarmony core
Primary Language	Java, Kotlin	Swift, Objective-C	ArkTS, JavaScript/TypeScript
Global Market Share	~71%	~28%	~4% (3rd globally, 2nd in China)

The Three Major Players — Overview

Key Insight:

- **Android** = Open, customizable, runs on thousands of devices
- **ioS** = Closed, polished, seamless Apple ecosystem
- **HarmonyOS** = Distributed, IoT-native, multi-device collaboration

Historical Context — How We Got Three OSes

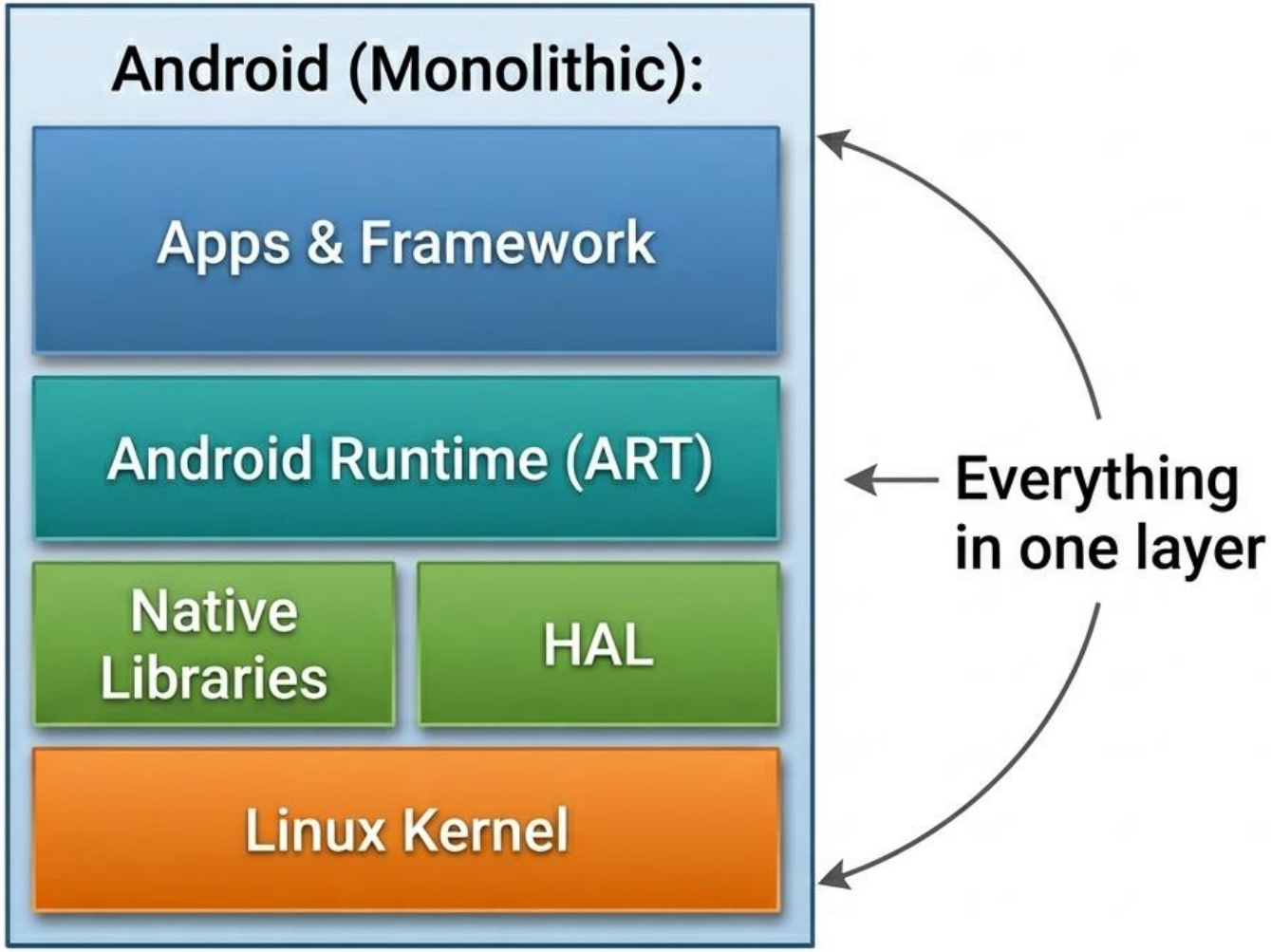
Year	Milestone	Significance
2007	iPhone + iOS launch	First true smartphone OS — touch-first, app ecosystem
2008	Android 1.0 release	Open alternative to iOS — Google + Open Handset Alliance
2008-2018	Android-iOS duopoly	Two ecosystems dominate globally
2019	US sanctions on Huawei	Cannot use Google Mobile Services — forces OS independence
2019	HarmonyOS announced	Huawei's answer — distributed OS for "1+8+N" strategy
2021+	HarmonyOS 2.0+ rollout	Rapid growth in China, expanding device categories
2024+	HarmonyOS NEXT	"Pure blood" version — no Android compatibility, fully native

Architecture Deep Dive — Kernel Design

Architecture Diagrams:

A **monolithic kernel** is a kernel design where **all core operating system services run in the same kernel space**.

This means the following components run inside the kernel itself:(**Process management, Memory management, File systems, Device drivers, Network stack**)

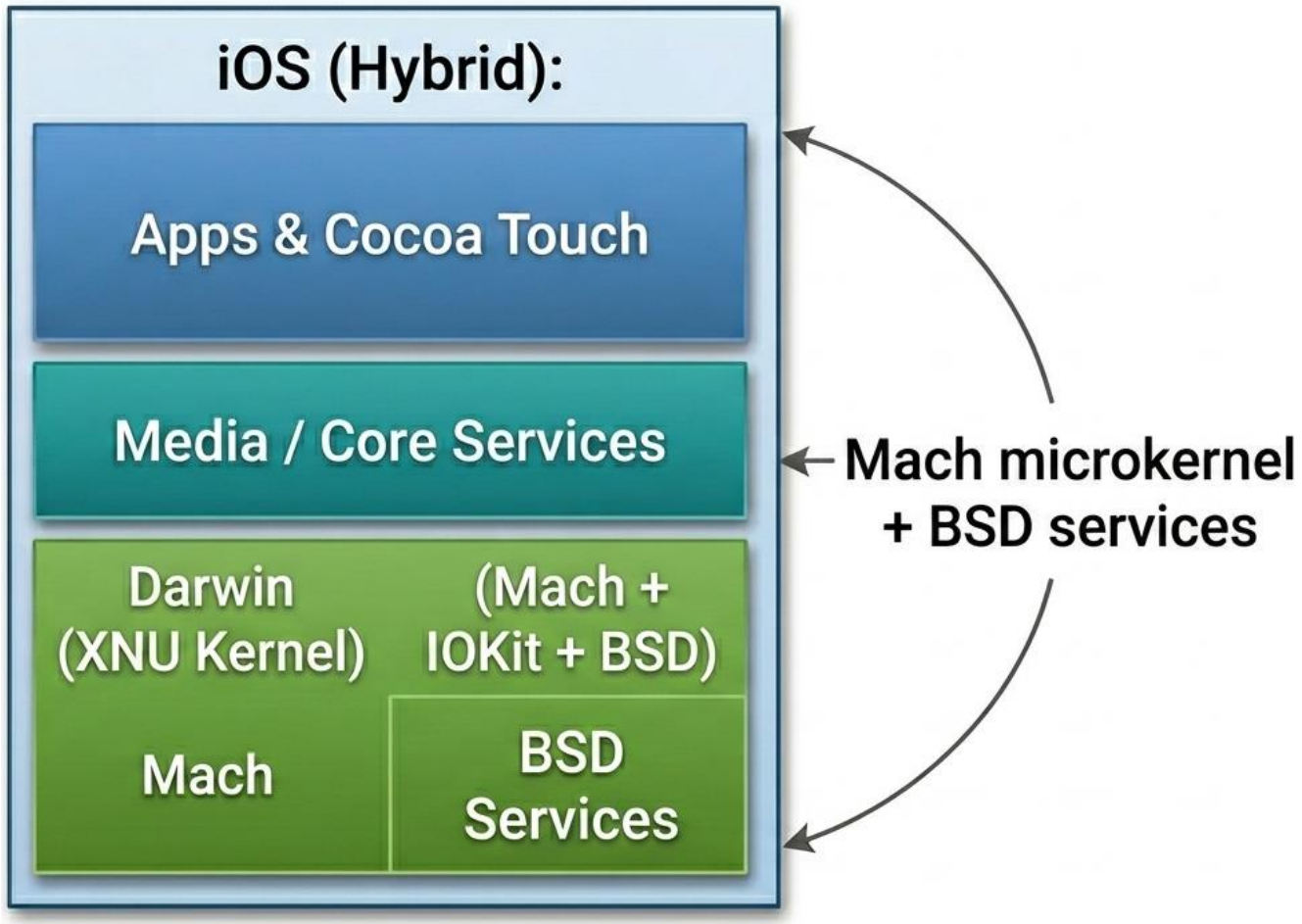


Architecture Deep Dive — Kernel Design

Architecture Diagrams:

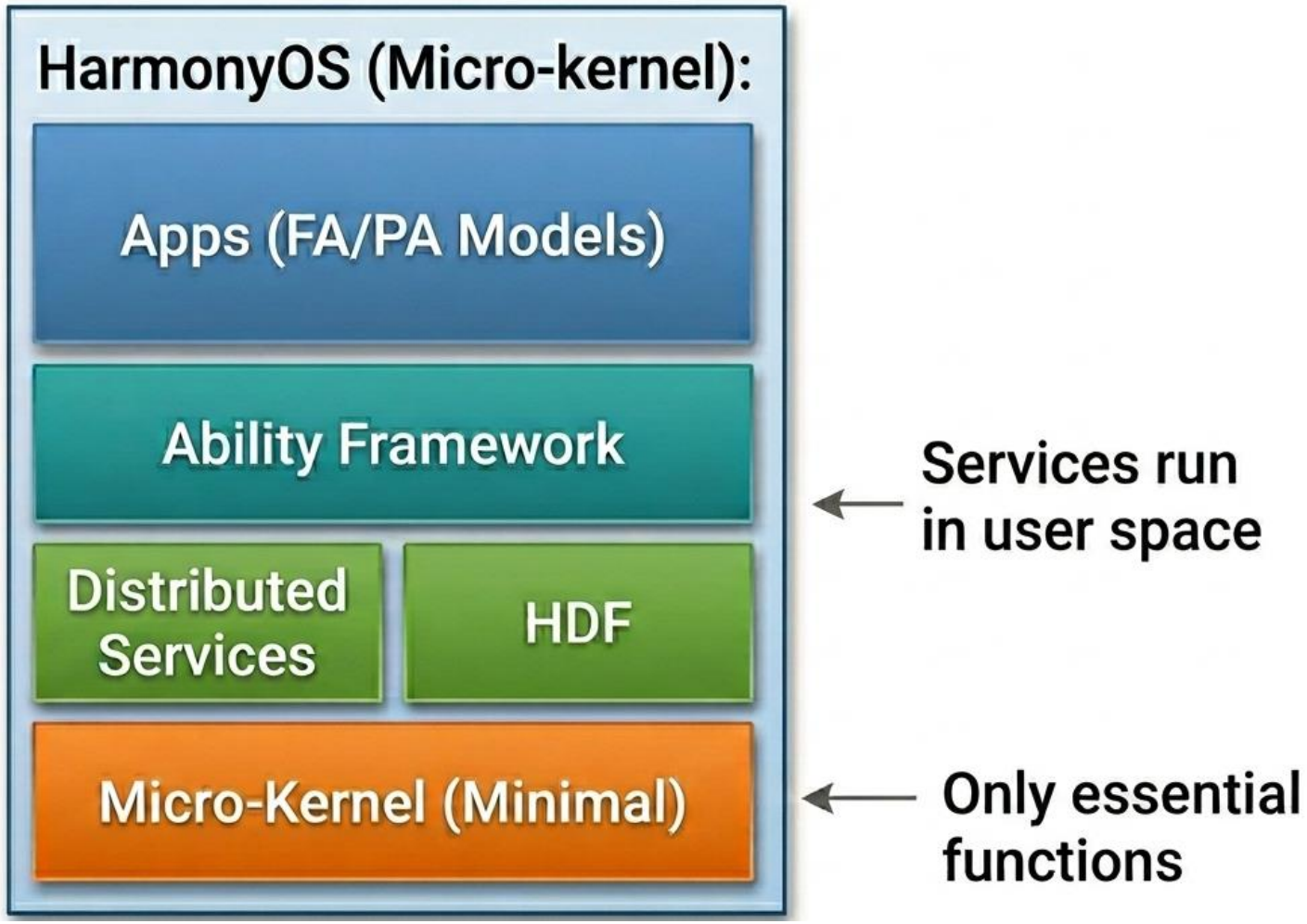
The operating system **iOS** uses the **XNU kernel**, which stands for **"X is Not Unix."** XNU is considered a **hybrid kernel** because it combines features of two kernel architectures:

- **Microkernel**
- **Monolithic kernel**



Architecture Deep Dive — Kernel Design

Architecture Diagrams:



Open vs. Closed Systems

The Fundamental Divide:

Characteristic	**Open (Android/HarmonyOS)**	**Closed (iOS)**
Source Code	Publicly available	Proprietary, secret
Device Variety	Thousands of manufacturers	Apple only
Customization	Manufacturers can modify UI/UX	Uniform experience
App Distribution	Multiple stores possible	App Store only
Hardware Access	More flexible	Controlled, consistent
Security Trade-off	More attack surface (fragmentation)	Tighter control, faster updates

Open vs. Closed Systems

Android Openness:

- **AOSP (Android Open-Source Project):** Base code free for anyone
- **OEM Customizations:** Samsung One UI, Xiaomi MIUI, etc.
- **Challenge:** Fragmentation — different versions, update delays

Open vs. Closed Systems

iOS Closedness:

- **Vertical Integration:** Apple designs hardware + software together
- **Benefit:** Optimized performance, consistent security updates
- **Trade-off:** No hardware choice, premium pricing

Open vs. Closed Systems

HarmonyOS Approach:

- **OpenHarmony:** Core system open source
- **Commercial HarmonyOS:** Huawei's enhanced version
- **Strategy:** Open core, but controlled ecosystem for quality

The Android Ecosystem

The Android Ecosystem

Philosophy: *"Billions of devices, one platform"*
Key Characteristics:

Aspect	Details
Developer	Google (Alphabet Inc.)
Base	Linux kernel + Android Runtime (ART)
Languages	Java, Kotlin (official), C++ (NDK)
App Store	Google Play Store (3.5M+ apps)
Distribution	AOSP + Google Mobile Services (GMS)
Device Range	$50 entry-level to $2,000+ flagships

The Android Ecosystem

Versions & Fragmentation Challenge:

- **Latest**: Android 15 (2024)
- **Problem**: Manufacturers customize and delay updates
- **Result**: Thousands of active Android versions in the wild

The Android Ecosystem

Strengths:

- Hardware choice at every price point
- Deep Google service integration (Search, Maps, Assistant)
- Customization and personalization
- Largest global market share (~71%)

The Android Ecosystem

Weaknesses:

- Security update inconsistency
- Quality variance across devices
- App compatibility testing complexity

The iOS Ecosystem

Seamless Integration, Fluid Experience, One Account, Shared Future

The iOS Ecosystem

Philosophy: "It just works"

Key Characteristics:

Aspect	Details
Developer	Apple Inc.
Base	Darwin/XNU kernel + Cocoa Touch framework
Languages	Swift (modern), Objective-C (legacy)
App Store	Apple App Store (2.2M+ apps, curated)
Devices	iPhone, iPad, iPod Touch only
Integration	Seamless with Mac, Watch, AirPods, Vision Pro

The iOS Ecosystem

The Apple Advantage:

Feature	Implementation
Hardware-Software Optimization	Custom A-series/M-series chips designed for iOS
Security	Secure Enclave, biometric authentication, app review
Updates	80%+ adoption of latest version within months
Ecosystem Lock-in	Handoff, AirDrop, Continuity across devices

The iOS Ecosystem

Strengths:

- Consistent, polished user experience
- Industry-leading security and privacy
- Long-term software support (5-6 years)
- Premium app quality and monetization

The iOS Ecosystem

Weaknesses:

- No hardware choice (only Apple devices)
- Premium pricing ($400-$1,500+)
- Limited customization
- 30% App Store commission controversial

The HarmonyOS Ecosystem

The HarmonyOS Ecosystem

The HarmonyOS Ecosystem

Philosophy: *"1+8+N" — One phone, eight core devices, N IoT partners*

Aspect	Details
Developer	Huawei Technologies
Base	Distributed micro-kernel (Hongmeng/LiteOS)
Languages	ArkTS (preferred), JavaScript/TypeScript, C/C++
App Store	Huawei AppGallery (15,000+ native HarmonyOS apps)
Devices	Phones, tablets, watches, TVs, cars, IoT
Unique Feature	**Distributed Soft Bus** — devices collaborate as one

The HarmonyOS Ecosystem

HarmonyOS NEXT (2024+):

- "Pure blood" — completely independent of Android
- No APK compatibility — pure native HarmonyOS app
- Strategy: Force ecosystem maturation through exclusivity

The HarmonyOS Ecosystem

Strengths:

- Native multi-device collaboration (unique in market)
- Deterministic latency for real-time applications
- IoT-first design from ground up
- Strong in China, growing globally

The HarmonyOS Ecosystem

Weaknesses:

- App ecosystem still maturing (vs. 3.5M Android / 2.2M iOS apps)
- Limited global device availability (Huawei sanctions impact)
- Developer migration effort required

Side-by-Side Comparison — Key Features

Feature	**Android**	**iOS**	**HarmonyOS**
Multi-Device Support	Cast, Nearby Share, limited	Handoff, Continuity, AirDrop	**Distributed Soft Bus native, automatic**
Device Discovery	Manual pairing	Manual within ecosystem	**Automatic (<20ms), any device type**
Hardware Resource Sharing	App-to-app with permissions	Apple ecosystem only	**Any device, any resource system native**
App Continuation	Limited	Seamless across Apple devices	**Seamless across any HarmonyOS device**
Development Approach	Write once, run many (fragmentation)	Write once, run Apple (controlled)	**Write once, run on 1+8+N (distributed)**
AI Integration	Google Assistant, Gemini, on-device ML	Siri, Core ML, Neural Engine	**Pangu model, XiaoYi, system-level AI**
Security Architecture	Sandboxing + Play Protect	Sandboxing + hardware security	

Side-by-Side Comparison — Key Features

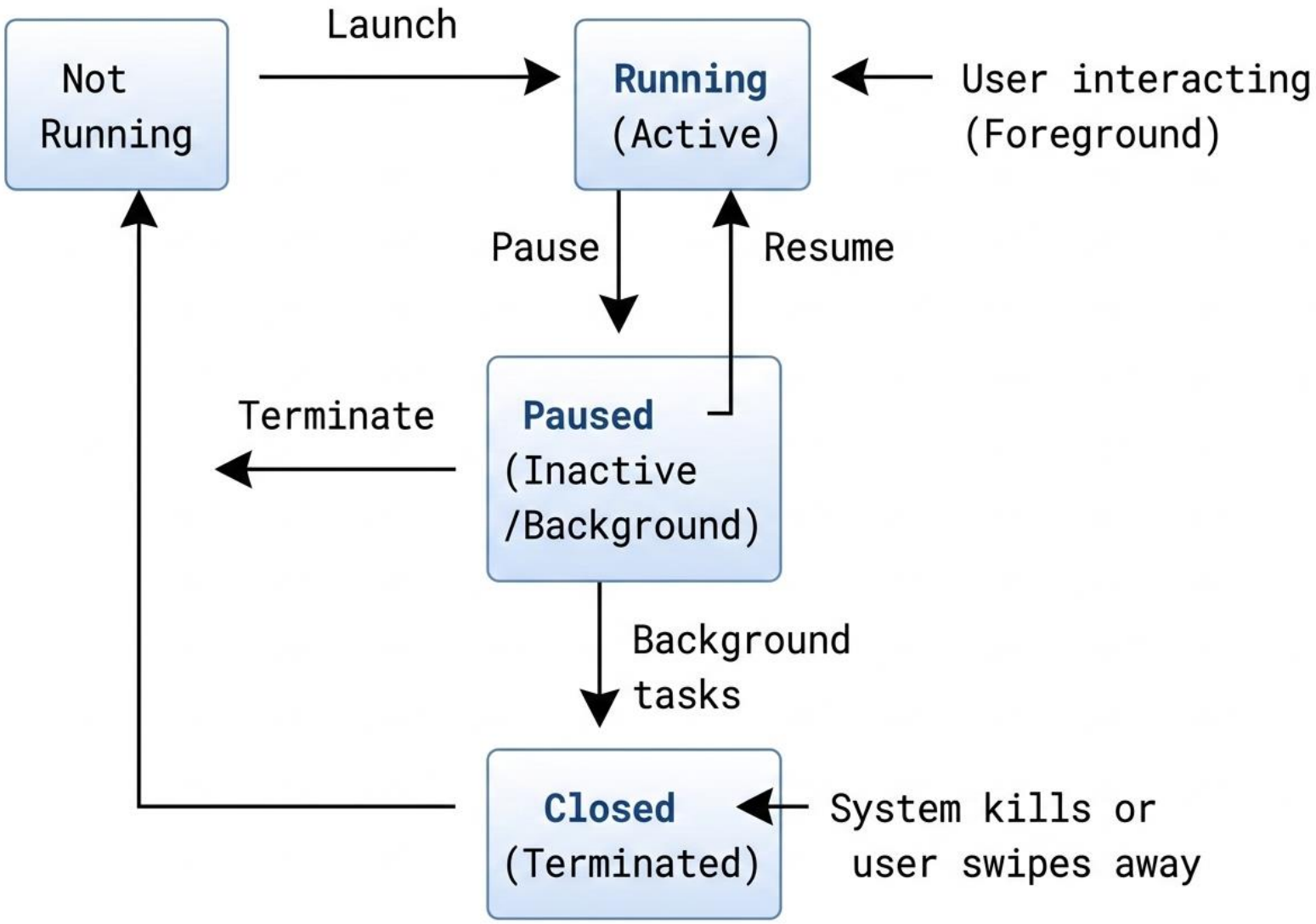
Key Differentiator Summary:

- **Android:** Maximum flexibility, runs on anything
- **iOS:** Maximum polish, works seamlessly within Apple
- **HarmonyOS: Maximum distribution — devices become one super-device**

App Marketplaces & Distribution

Aspect	**Google Play Store**	**Apple App Store**	**Huawei AppGallery**
Developer Fee	$25 one-time	$99/year	Free registration
Review Process	Automated, less restrictive	Manual, 2-3 days, strict	Curated, growing standards
App Count	~3.5 million	~2.2 million	~15,000 native HarmonyOS
Revenue Share	70/30 (85/15 first $1M)	70/30 (85/15 small devs)	Competitive, region-dependent
Global Reach	190+ countries	175+ countries	170+ countries, strong China
Key Strength	Volume, reach	Quality, monetization	China market, growth incentives

Mobile App Lifecycle — The States



Mobile App Lifecycle — The States

State Descriptions:

State	User Visibility	System Resources	Example
Running/Active	On screen, interacting	Full CPU, memory, battery	Playing game, typing message
Paused/Inactive	On screen but interrupted	Reduced resources	Incoming call during game
Background	Not visible	Limited resources, may be suspended	Music playing, download continuing
Closed/Terminated	Not in memory	No resources	App swiped away, system killed

Development Models — Building for Each OS

Aspect	**Android**	**iOS**	**HarmonyOS**
Official IDE	Android Studio	Xcode	DevEco Studio
Primary Languages	Kotlin, Java	Swift	ArkTS, JS/TS
UI Framework	Jetpack Compose, XML	SwiftUI, UIKit	ArkUI (declarative)
Build System	Gradle	Xcode build system	Hvigor
Emulator	Android Emulator	iOS Simulator	HarmonyOS Emulator
Testing Devices	Wide hardware variety needed	Limited Apple devices	Huawei devices + distributed testing

ADAMA SCIENCE & TECHNOLOGY UNIVERSITY

END *of* CHAPTER 2